

SIRO HR Chatbot Dockerization ve Deployment Dokümanı

SIRO HR Chatbot Dockerization ve Deployment Dokümanı.....	1
1. Dokümanın Amacı.....	3
2. Genel Sistem Mimarisi.....	3
Frontend Container.....	3
Backend Container.....	4
MySQL Container.....	4
3. Güncel Proje Klasör Yapısı.....	5
4. Environment Dosyaları.....	6
4.1 Deployment Repo .env Dosyası.....	7
4.2 Deployment Repo .env.example Dosyası.....	7
4.3 Backend.env Dosyası.....	8
5. Backend Dockerization.....	9
Backend Dockerfile.....	10
Backend Dockerfile Açıklaması.....	10
6. Backend .dockerignore Dosyası.....	11
7. Frontend Dockerization.....	13
Frontend Dockerfile.....	13
Frontend Dockerfile Açıklaması.....	14
8. Frontend.dockerignore Dosyası.....	15
9. MySQL Dockerization.....	16
MySQL Service Özeti.....	16
Neden MySQL Container Eklendi?.....	16
10. MySQL init.sql Dosyası.....	17
Production İçin Seed Data Notu.....	18
11. Docker Compose Yapısı.....	18
12. Docker Compose Açıklaması.....	20
12.1 MySQL Servisi.....	20
12.2 Backend Servisi.....	22
12.3 Frontend Servisi.....	24
13. Local Ortamda Çalıştırma Adımları.....	25
Ön Koşullar.....	25
Adım 1: Repo klasör yapısı hazırlanır.....	26
Adım 2: Backend .env dosyası hazırlanır.....	26
Adım 3: Deployment.....	27

.env dosyası hazırlanır.....	27
Adım 4: Docker Compose config kontrolü yapılır.....	27
Adım 5: İlk temiz başlatma yapılır.....	27
Adım 6: Normal başlatma.....	28
Adım 7: Uygulama kontrol edilir.....	28
Adım 8: Container'lar kontrol edilir.....	29
Adım 9: Loglar kontrol edilir.....	29
Adım 10: Sistemi durdurma.....	30
14. Kalıcı Veri ve Volume Kullanımı.....	30
14.1 uploaded_documents.....	31
14.2 faiss_index_3b.....	31
14.3 mysql_data.....	32
15. Production Deployment Yaklaşımı.....	32
Production .env Ayarı.....	33
Production MySQL Ayarı.....	33
Production Kullanıcı ve Admin Yönetimi.....	34
Reverse Proxy Mantiği.....	35
16. CPU ve GPU Değerlendirmesi.....	36
17. Kullanışlı Docker Komutları.....	36
18. Karşılaşılan Hatalar ve Çözümleri.....	39
Hata: docker: command not found.....	39
Hata: Eksik Python paketi.....	39
Hata: Frontend TypeScript build hatası.....	39
Hata: Port zaten kullanımda.....	40
Hata: Container name conflict.....	41
Hata: Backend MySQL'e bağlanamıyor.....	41
Hata: init.sql değişti ama database güncellenmedi.....	42
Hata: Frontend backend'e erişemiyor.....	43
Hata: Model yükleme çok uzun sürüyor.....	44
19. Bakım ve Güncelleme Süreci.....	44
Backend kodu güncellendiğinde.....	44
Frontend kodu güncellendiğinde.....	44
Backend API URL değiştirildiğinde.....	45
Backend.env değiştiğinde.....	45
MySQL schema değiştiğinde.....	45
Dokümanlar güncellendiğinde.....	46
FAISS index güncellendiğinde.....	46
20. Final Kontrol Listesi.....	46
21. Teslim Notu.....	48

1. Dokümanın Amacı

Bu dokümanın amacı, SIRO HR Chatbot uygulamasının Docker ile nasıl container yapısına alındığını ve local veya production ortamında nasıl deploy edilebileceğini açıklamaktır.

SIRO HR Chatbot projesi üç ana bileşenden oluşmaktadır:

1. **Backend:** FastAPI tabanlı Python uygulamasıdır. Kullanıcı doğrulama, chat yönetimi, admin işlemleri, doküman yükleme, kategori kısıtlamaları, SAP/HR API entegrasyonu ve RAG pipeline işlemleri backend tarafında yürütülür.
2. **Frontend:** React, TypeScript ve Vite ile geliştirilmiş kullanıcı arayüzüdür. Kullanıcı login ekranı, chat ekranı, admin paneli, doküman yönetimi ve kategori kısıtlama ekranları frontend üzerinden sunulur.
3. **MySQL Database:** Kullanıcı, admin, chat geçmişi, chatbot kategori ayarları, doküman metadata bilgileri ve upload kayıtlarının tutulduğu ilişkisel veritabanıdır.

Önceki yapıda backend ve frontend container'ları Docker Compose ile birlikte çalıştırılabiliyordu. Güncel deployment yapısında MySQL de Docker Compose içine eklenmiştir. Böylece sistem yalnızca backend ve frontend'den oluşan bir yapı olmaktan çıkarılmış, veritabanı dahil daha bütünlüklü ve tekrarlanabilir bir deployment yapısına dönüştürülmüştür.

Proje klasörleri local ortamda şu şekilde konumlandırılmıştır:

None

```
~/Desktop/siro_hr_chatbot          → Backend uygulaması
~/Desktop/siro_chatbot_FE          → Frontend uygulaması
~/Desktop/siro-hr-chatbot-deployment → Docker Compose ve
deployment dosyaları
```

Bu yapı sayesinde backend, frontend ve MySQL servisleri tek bir Docker Compose komutu ile birlikte ayağa kaldırılabilir.

2. Genel Sistem Mimarisi

Güncel Dockerized sistem üç ana container'dan oluşmaktadır:

Frontend Container

React/Vite uygulamasının build çıktısını Nginx üzerinden servis eder.

- Host port: 3000

- Container port: 80
- Local erişim adresi: <http://localhost:3000>

Backend Container

FastAPI uygulamasını Uvicorn ile çalıştırır.

- Host port: 8000
- Container port: 8000
- Local erişim adresi: <http://localhost:8000>
- Swagger/OpenAPI adresi: <http://localhost:8000/docs>

MySQL Container

Uygulamanın ilişkisel veritabanını çalıştırır.

- Image: [mysql:8.0](#)
- Container adı: [siro-hr-mysql](#)
- Container internal port: 3306
- Host port: 3307
- Backend bağlantı hostname'i: [mysql](#)
- Kalıcı volume: [mysql_data](#)

Local çalışma ortamında sistem akışı şu şekildedir:

```
None
Kullanıcı Tarayıcısı
  ↓
Frontend Container
http://localhost:3000
  ↓ API istekleri
Backend Container
http://localhost:8000
  ↓
MySQL Container
mysql:3306
```

Backend aynı zamanda RAG pipeline, FAISS index, yüklenen dokümanlar, kategori sınıflandırma modeli, embedding/reranking modelleri, LLM ve dış SAP/HR API entegrasyonları ile çalışır.

Production ortamında kullanıcıların `localhost` adresi görmesi beklenmez. Production deployment sırasında bu adresler domain veya subdomain ile değiştirilmelidir. Örneğin:

None

```
Frontend: https://chatbot.siro.com
Backend:  https://api-chatbot.siro.com
```

veya tek domain altında:

None

```
Frontend: https://chatbot.siro.com
Backend:  https://chatbot.siro.com/api
```

Bu dönüşüm DNS, reverse proxy ve SSL ayarları ile yapılır.

3. Güncel Proje Klasör Yapısı

Güncel local deployment klasör yapısı aşağıdaki gibidir:

None

```
~/Desktop/
|
├─ siro_hr_chatbot/
|   ├── Dockerfile
|   ├── .dockerignore
|   ├── .env
|   ├── requirements.txt
|   ├── app.py
|   ├── db.py
|   ├── rag/
|   ├── authentication/
|   ├── uploaded_documents/
|   └─ faiss_index_3b/
|
└─ siro_chatbot_FE/
```

```
|   ├── Dockerfile
|   ├── .dockerignore
|   ├── package.json
|   ├── package-lock.json
|   ├── src/
|   ├── public/
|   └── vite.config.ts
└── siro-hr-chatbot-deployment/
    ├── docker-compose.yml
    ├── .env
    ├── .env.example
    ├── .gitignore
    ├── README.md
    ├── docs/
    │   └── deployment-guide.md
    └── mysql/
        └── init.sql
```

Bu yapıda deployment dosyaları backend ve frontend projelerinden ayrı bir repoda tutulur. Bunun nedeni deployment konfigürasyonunu merkezi ve temiz bir şekilde yönetmektir.

Deployment reposu, backend ve frontend repolarıyla aynı üst klasörde bulunmalıdır. Çünkü `docker-compose.yml` içinde backend ve frontend build context yolları şu şekilde verilmiştir:

None

```
../siro_hr_chatbot
../siro_chatbot_FE
```

Bu nedenle deployment repo farklı bir konuma alınırsa `docker-compose.yml` içindeki context path'lerinin de güncellenmesi gerekir.

4. Environment Dosyaları

Sistemde iki ayrı `.env` dosyası bulunmaktadır. Bunların görevleri birbirinden farklıdır.

4.1 Deployment Repo **.env** Dosyası

Konum:

None

siro-hr-chatbot-deployment/.env

Bu dosya Docker Compose tarafından okunur. Frontend build sırasında kullanılacak backend API adresi ve MySQL container ayarları burada tutulur.

Local test için örnek içerik:

None

VITE_CHATBOT_BACKEND_API=http://localhost:8000

MYSQL_ROOT_PASSWORD=123456

MYSQL_DATABASE=chatbot_db

Bu dosya Git'e pushlanmamalıdır. Bu nedenle deployment repo **.gitignore** içinde **.env** yer almaktadır.

4.2 Deployment Repo **.env.example** Dosyası

Konum:

None

siro-hr-chatbot-deployment/.env.example

Bu dosya gerçek secret içermez. Yeni kurulum yapacak kişilere hangi environment değişkenlerinin gerektiğini göstermek için kullanılır.

İçeriği:

None

VITE_CHATBOT_BACKEND_API=http://localhost:8000

MYSQL_ROOT_PASSWORD=your_mysql_root_password

MYSQL_DATABASE=chatbot_db

Yeni bir geliştirici deployment reposunu clone'ladıktan sonra şu komutla kendi local `.env` dosyasını oluşturabilir:

None

```
cp .env.example .env
```

Daha sonra `.env` içindeki `MYSQL_ROOT_PASSWORD` değeri backend `.env` içindeki `DB_PASSWORD` değeriyle aynı yapılmalıdır.

4.3 Backend `.env` Dosyası

Konum:

None

```
siro_hr_chatbot/.env
```

Bu dosya backend container'a runtime sırasında verilir. Backend'in ihtiyaç duyduğu database bilgileri, JWT secret, SAP API bilgileri, frontend callback/origin ayarları ve benzeri runtime konfigürasyonlar burada tutulur.

Docker Compose içindeki MySQL servisine bağlanmak için backend `.env` dosyasındaki database kısmı şu şekilde olmalıdır:

None

```
DB_HOST=mysql
DB_PORT=3306
DB_USER=root
DB_PASSWORD=123456
DB_NAME=chatbot_db
```

Burada `DB_HOST=mysql` değeri önemlidir. Backend container, MySQL container'a Docker Compose network'ü üzerinden `mysql` servis adıyla erişir. Bu nedenle Docker içinde `DB_HOST=localhost` kullanılmamalıdır. Container içindeki `localhost`, host makineyi veya MySQL container'ı değil, backend container'ın kendisini ifade eder.

Backend `.env` dosyasındaki `DB_PASSWORD`, deployment repo `.env` dosyasındaki `MYSQL_ROOT_PASSWORD` ile aynı olmalıdır. Aksi halde backend MySQL'e bağlanamaz.

Örnek uyumlu yapı:

Deployment `.env`:

```
None
MYSQL_ROOT_PASSWORD=123456
MYSQL_DATABASE=chatbot_db
```

Backend `.env`:

```
None
DB_HOST=mysql
DB_PORT=3306
DB_USER=root
DB_PASSWORD=123456
DB_NAME=chatbot_db
```

5. Backend Dockerization

Backend uygulaması FastAPI ile geliştirilmiştir. Uygulamanın giriş dosyası:

```
None
app.py
```

FastAPI instance adı:

```
None
app
```

Bu nedenle backend container içinde uygulama şu komutla çalıştırılır:

```
None
uvicorn app:app --host 0.0.0.0 --port 8000
```

Burada `--host 0.0.0.0` kullanılması önemlidir. Çünkü container içindeki uygulamanın container dışından erişilebilir olması için yalnızca `127.0.0.1` üzerinde değil, tüm network interface'leri üzerinde dinlemesi gerekir.

Backend Dockerfile

Konum:

None

siro_hr_chatbot/Dockerfile

İçerik:

None

```
FROM python:3.11-slim
```

```
WORKDIR /app
```

```
ENV PYTHONDONTWRITEBYTECODE=1
```

```
ENV PYTHONUNBUFFERED=1
```

```
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential \
    curl \
    && rm -rf /var/lib/apt/lists/*
```

```
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir --upgrade pip && \
    pip install --no-cache-dir -r requirements.txt
```

```
COPY . .
```

```
EXPOSE 8000
```

```
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

Backend Dockerfile Açıklaması

```
FROM python:3.11-slim
```

Backend için hafif bir Python imajı kullanılır. `slim` imaj tam Python imajına göre daha küçüktür ve gereksiz sistem bileşenlerini içermez.

```
WORKDIR /app
```

Container içindeki çalışma dizini `/app` olarak belirlenir.

```
ENV PYTHONDONTWRITEBYTECODE=1
```

Python'un gereksiz `.pyc` dosyaları üretmesini engeller.

```
ENV PYTHONUNBUFFERED=1
```

Logların gecikmeden terminale düşmesini sağlar. Docker log takibi için önemlidir.

```
RUN apt-get update ...
```

Bazı Python paketlerinin kurulumu sırasında build araçlarına ihtiyaç duyulabilir. Bu nedenle `build-essential` kurulmuştur. `curl` ise gerektiğinde bağlantı veya health check işlemlerinde kullanılabilir.

```
COPY requirements.txt .
```

Önce yalnızca `requirements.txt` kopyalanır. Bu yaklaşım Docker cache mekanizmasını daha verimli kullanmak için tercih edilir.

```
RUN pip install ...
```

Python bağımlılıkları container içine kurulur.

```
COPY . .
```

Backend proje dosyaları container içine kopyalanır.

```
EXPOSE 8000
```

Backend uygulamasının container içinde 8000 portundan çalıştığı belirtilir.

```
CMD ["uvicorn", ...]
```

Container başlatıldığında FastAPI uygulaması Uvicorn ile çalıştırılır.

6. Backend `.dockerignore` Dosyası

Konum:

```
None  
siro_hr_chatbot/.dockerignore
```

İçerik:

```
None  
venv  
__pycache__  
*.pyc  
.git  
.gitignore  
.DS_Store  
  
node_modules  
package-lock.json  
package.json  
  
.env  
  
chroma_db
```

`.dockerignore` dosyası, Docker image oluşturulurken gereksiz veya hassas dosyaların image içine kopyalanmasını engeller.

Özellikle şu dosya ve klasörlerin image içine alınmaması önemlidir:

- `venv`: Local Python virtual environment container içine kopyalanmamalıdır.
- `__pycache__` ve `*.pyc`: Python cache dosyaları gereksizdir.
- `.git`: Git geçmişi Docker image içine alınmamalıdır.
- `.env`: Gizli bilgiler image içine gömülmemelidir.
- `node_modules`, `package.json`, `package-lock.json`: Backend container için gerekli değildir.
- `chroma_db`: Bu deployment yapısında FAISS kullanıldığı için image içine alınmamıştır.

Backend `.env` dosyası image içine kopyalanmaz. Bunun yerine Docker Compose üzerinden runtime sırasında container'a verilir.

7. Frontend Dockerization

Frontend uygulaması React, TypeScript ve Vite ile geliştirilmiştir.

Frontend build komutu:

```
None  
npm run build
```

Bu komut çalıştırıldığında statik build çıktısı `dist/` klasörü altında oluşur. Production ortamında frontend uygulamasını Vite dev server ile çalıştırmak yerine, build çıktısını Nginx üzerinden servis etmek daha doğru bir yaklaşımdır.

Bu nedenle frontend Dockerfile iki aşamalı, yani multi-stage build olarak hazırlanmıştır:

1. Node.js aşaması ile uygulama build edilir.
2. Nginx aşaması ile build çıktısı servis edilir.

Frontend Dockerfile

Konum:

```
None  
siro_chatbot_FE/Dockerfile
```

İçerik:

```
None  
FROM node:20-alpine AS build  
  
WORKDIR /app  
  
ARG VITE_CHATBOT_BACKEND_API  
ENV VITE_CHATBOT_BACKEND_API=$VITE_CHATBOT_BACKEND_API  
  
COPY package*.json ./
```

```
RUN npm ci

COPY . .

RUN npm run build

FROM nginx:alpine

COPY --from=build /app/dist /usr/share/nginx/html

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

Frontend Dockerfile Açıklaması

```
FROM node:20-alpine AS build
```

İlk aşamada Node.js imajı kullanılır. Bu aşama yalnızca frontend projesini build etmek içindir.

```
ARG VITE_CHATBOT_BACKEND_API
```

Frontend'in backend API adresi build argümanı olarak alınır.

```
ENV VITE_CHATBOT_BACKEND_API=$VITE_CHATBOT_BACKEND_API
```

Build argümanı environment değişkenine atanır. Vite projelerinde `import.meta.env` ile kullanılan environment değişkenleri build sırasında frontend koduna gömülür. Bu nedenle backend API adresi container çalışırken değil, frontend build edilirken verilmelidir.

Frontend kodunda backend API adresi şu şekilde okunmaktadır:

```
None

export const API_BASE_URL =
import.meta.env.VITE_CHATBOT_BACKEND_API;
```

```
RUN npm ci
```

`package-lock.json` dosyasına göre temiz ve tekrarlanabilir bağımlılık kurulumu yapar.

```
RUN npm run build
```

Production build alınır.

```
FROM nginx:alpine
```

İkinci aşamada hafif bir Nginx imajı kullanılır.

```
COPY --from=build /app/dist /usr/share/nginx/html
```

Build çıktısı Nginx'in statik dosya servis ettiği dizine kopyalanır.

```
EXPOSE 80
```

Container içinde Nginx 80 portundan çalışır.

8. Frontend **.dockerignore** Dosyası

Konum:

```
None  
siro_chatbot_FE/.dockerignore
```

İçerik:

```
None  
node_modules  
dist  
.git  
.gitignore  
.DS_Store  
.env
```

Bu dosya sayesinde local bağımlılıklar, build çıktıları, Git metadata dosyaları ve environment dosyaları Docker image içine alınmaz.

Özellikle:

- `node_modules`: Container içinde yeniden kurulacağı için local `node_modules` kopyalanmaz.
 - `dist`: Build çıktısı container içinde yeniden üretileceği için kopyalanmaz.
 - `.env`: Environment değerleri image içine gömülmemelidir.
-

9. MySQL Dockerization

Güncel deployment yapısında MySQL veritabanı da Docker Compose içine alınmıştır. Böylece geliştiricilerin veya test edecek kişilerin kendi bilgisayarlarına ayrı bir MySQL kurulumu yapması gerekmez.

MySQL servisi Docker Compose içinde `mysql` adıyla tanımlanmıştır. Backend container, MySQL container'a bu servis adı üzerinden bağlanır.

MySQL Service Özeti

- Image: `mysql:8.0`
- Container adı: `siro-hr-mysql`
- Internal port: `3306`
- Host port: `3307`
- Docker network hostname: `mysql`
- Persistent volume: `mysql_data`
- Init script: `./mysql/init.sql`

Host port olarak `3307` kullanılmıştır. Bunun nedeni, geliştirici makinesinde halihazırda local MySQL çalışıyorsa 3306 port çakışmasını önlemektir. Backend container ise Docker network içinde MySQL'e `mysql:3306` adresinden bağlanır.

Neden MySQL Container Eklendi?

Backend uygulaması MySQL tablolarını kendi kendine otomatik oluşturmamaktadır. Kod içinde `CREATE TABLE`, migration veya ORM tabanlı `create_all` mekanizması bulunmadığı için backend hazır database ve tablolar beklemektedir.

Bu nedenle deployment tarafında iki ihtiyaç ortaya çıkmıştır:

1. MySQL servisinin Docker Compose ile otomatik ayağa kaldırılması.
2. Gerekli database schema'sının MySQL container ilk oluşturulduğunda otomatik hazırlanması.

Bu ihtiyaçlar için `mysql/init.sql` dosyası oluşturulmuştur.

10. MySQL `init.sql` Dosyası

Konum:

None

`siro-hr-chatbot-deployment/mysql/init.sql`

Bu dosya MySQL container ilk kez oluşturulduğunda otomatik çalıştırılır. Docker MySQL image'ı, `/docker-entrypoint-initdb.d/` klasörüne bağlanan `.sql` dosyalarını ilk database initialization sırasında çalıştırır.

Docker Compose içinde bu bağlantı şu şekilde yapılmıştır:

None

`volumes:`

- `mysql_data:/var/lib/mysql`
- `./mysql/init.sql:/docker-entrypoint-initdb.d/init.sql`

`init.sql` dosyasının görevleri:

1. `chatbot_db` database'ini oluşturmak.
2. Gerekli tabloları oluşturmak.
3. Foreign key ilişkilerini kurmak.
4. Chatbot kategori ayarları için başlangıç kategori verilerini eklemek.

Oluşturulan tablolar:

- `users`
- `admins`
- `admin_accounts`
- `employees`
- `employee_accounts`
- `chats`
- `chat_messages`
- `chatbot_categories`
- `documents`

- `uploads`

Production İçin Seed Data Notu

`init.sql` içinde test kullanıcıları veya test admin hesapları bulundurmak production ortamı için doğru değildir. Kullanıcı hesaplarının SAP/SSO entegrasyonu üzerinden sağlanması, admin yetkilerinin ise şirketin belirleyeceği authorization politikasına göre atanması beklenir.

Bu nedenle production tesliminde `init.sql` dosyasının yalnızca schema ve sistem konfigürasyonu için gerekli başlangıç verilerini içermesi önerilir.

Production'a uygun `init.sql` içinde kalması mantıklı olan veri:

- `chatbot_categories` başlangıç kayıtları

Production'a uygun olmayan test verileri:

- `tarik@siro.com`
- `tarik@admin.com`
- `dummy` password hash değerleri
- demo employee/admin hesapları

Local demo/test ortamında test kullanıcıları gerekiyorsa bunlar ayrı bir dosyada tutulabilir:

None

`mysql/sample_seed.sql`

Bu dosya otomatik çalıştırılmaz. Gerekirse manuel olarak çalıştırılır. Böylece production schema temiz kalırken demo/test kolaylığı da korunur.

11. Docker Compose Yapısı

Deployment için ana dosya:

None

`siro-hr-chatbot-deployment/docker-compose.yml`

Güncel Docker Compose yapısı üç servisten oluşmaktadır:

1. mysql
2. backend
3. frontend

Güncel içerik:

None

```
services:
  mysql:
    image: mysql:8.0
    container_name: siro-hr-mysql
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
      MYSQL_DATABASE: ${MYSQL_DATABASE}
    ports:
      - "3307:3306"
    volumes:
      - mysql_data:/var/lib/mysql
      - ./mysql/init.sql:/docker-entrypoint-initdb.d/init.sql
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-u",
"root", "-p${MYSQL_ROOT_PASSWORD}"]
      interval: 10s
      timeout: 5s
      retries: 10
      restart: unless-stopped

  backend:
    build:
      context: ../siro_hr_chatbot
    image: siro-hr-backend
    container_name: siro-hr-backend
    env_file:
      - ../siro_hr_chatbot/.env
    ports:
      - "8000:8000"
    volumes:
```

```
-  
../siro_hr_chatbot/uploaded_documents:/app/uploaded_documents  
- ../siro_hr_chatbot/faiss_index_3b:/app/faiss_index_3b  
depends_on:  
  mysql:  
    condition: service_healthy  
restart: unless-stopped  
  
frontend:  
  build:  
    context: ../siro_chatbot_FE  
    args:  
      VITE_CHATBOT_BACKEND_API: ${VITE_CHATBOT_BACKEND_API}  
  image: siro-chatbot-frontend  
  container_name: siro-chatbot-frontend  
  ports:  
    - "3000:80"  
  depends_on:  
    - backend  
  restart: unless-stopped  
  
volumes:  
  mysql_data:
```

12. Docker Compose Açıklaması

12.1 MySQL Servisi

MySQL servisi şu image üzerinden çalıştırılır:

```
None  
image: mysql:8.0
```

Container adı:

None

```
container_name: siro-hr-mysql
```

Environment değişkenleri deployment repo `.env` dosyasından alınır:

None

```
MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}  
MYSQL_DATABASE: ${MYSQL_DATABASE}
```

Local ortamda örnek değerler:

None

```
MYSQL_ROOT_PASSWORD=123456  
MYSQL_DATABASE=chatbot_db
```

Port mapping:

None

```
ports:  
  - "3307:3306"
```

Bu mapping, host makinede 3307 portunu MySQL container içindeki 3306 portuna bağlar. Böylece gerekirse host makineden MySQL'e şu bilgilerle bağlanılabilir:

None

```
Host: localhost  
Port: 3307  
User: root  
Password: MYSQL_ROOT_PASSWORD değeri  
Database: chatbot_db
```

Backend container ise host portunu kullanmaz. Backend, MySQL'e Docker network içinde şu şekilde erişir:

None

```
Host: mysql
```

Port: 3306

Volume tanımı:

None

volumes:

- mysql_data:/var/lib/mysql
- ./mysql/init.sql:/docker-entrypoint-initdb.d/init.sql

`mysql_data` volume'u veritabanı dosyalarını kalıcı hale getirir. `init.sql` ise MySQL container ilk kez initialize edilirken çalıştırılır.

Healthcheck:

None

healthcheck:

```
test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-u",  
"root", "-p${MYSQL_ROOT_PASSWORD}"]  
interval: 10s  
timeout: 5s  
retries: 10
```

Bu healthcheck sayesinde Docker Compose, MySQL servisi gerçekten hazır hale gelmeden backend'i başlatmaz.

12.2 Backend Servisi

Backend servisi şu proje klasöründen build edilir:

None

build:

context: ../siro_hr_chatbot

Oluşan image adı:

None

```
image: siro-hr-backend
```

Container adı:

None

```
container_name: siro-hr-backend
```

Backend environment dosyası şu şekilde verilir:

None

```
env_file:  
- ../siro_hr_chatbot/.env
```

Bu sayede backend'in ihtiyaç duyduğu gizli konfigürasyonlar image içine gömülmeden container'a aktarılır.

Port mapping:

None

```
ports:  
- "8000:8000"
```

Bu yapı sayesinde container içindeki 8000 portu host makinedeki 8000 portuna bağlanır. Backend local ortamda şu adresten erişilebilir:

None

```
http://localhost:8000
```

Swagger/OpenAPI arayüzü:

None

```
http://localhost:8000/docs
```

Backend için iki önemli klasör volume olarak bağlanmıştır:

None

volumes:

- ../siro_hr_chatbot/uploaded_documents:/app/uploaded_documents
- ../siro_hr_chatbot/faiss_index_3b:/app/faiss_index_3b

Bu sayede yüklenen dokümanlar ve FAISS index dosyaları container silinse bile host makinede kalmaya devam eder.

Backend'in MySQL'e bağımlılığı şu şekilde tanımlanmıştır:

None

depends_on:

mysql:

condition: service_healthy

Bu ayar, backend'in MySQL healthcheck başarılı olmadan başlatılmamasını sağlar.

12.3 Frontend Servisi

Frontend servisi şu proje klasöründen build edilir:

None

build:

context: ../siro_chatbot_FE

Frontend build sırasında backend API adresi şu build argümanı ile verilir:

None

args:

VITE_CHATBOT_BACKEND_API: \${VITE_CHATBOT_BACKEND_API}

Bu değer deployment repo **.env** dosyasından okunur.

Frontend port mapping:

None

ports:


```
- "3000:80"
```

Bu yapı sayesinde container içindeki Nginx'in 80 portu host makinedeki 3000 portuna bağlanır. Frontend local ortamda şu adresten erişilebilir:

```
None  
http://localhost:3000
```

Frontend servisi backend'e bağımlı olacak şekilde tanımlanmıştır:

```
None  
depends_on:  
- backend
```

Bu ayar Docker Compose'un backend container'ını frontend'den önce başlatmasını sağlar. Ancak bu basit dependency frontend'in backend'in tamamen hazır olmasını beklediği anlamına gelmez; yalnızca başlatma sırasını etkiler.

13. Local Ortamda Çalıştırma Adımları

Ön Koşullar

Makinede Docker kurulu ve çalışıyor olmalıdır.

Docker kurulumunu kontrol etmek için:

```
None  
docker --version
```

Docker Compose kontrolü için:

```
None  
docker compose version
```

Adım 1: Repo klasör yapısı hazırlanır

Üç repo aynı üst klasörde bulunmalıdır:

```
None
~/Desktop/
├─ siro_hr_chatbot/
├─ siro_chatbot_FE/
└─ siro-hr-chatbot-deployment/
```

Backend ve frontend repoları mevcutsa güncellenmelidir:

```
None
git pull
```

Deployment repo yoksa clone'lanmalıdır:

```
None
git clone <deployment-repo-url>
```

Adım 2: Backend **.env** dosyası hazırlanır

Backend klasöründe şu dosya bulunmalıdır:

```
None
siro_hr_chatbot/.env
```

Docker Compose içindeki MySQL servisine bağlanmak için database kısmı şu şekilde olmalıdır:

```
None
DB_HOST=mysql
DB_PORT=3306
DB_USER=root
DB_PASSWORD=123456
DB_NAME=chatbot_db
```

DB_PASSWORD, deployment repo **.env** içindeki **MYSQL_ROOT_PASSWORD** ile aynı olmalıdır.

Adım 3: Deployment

.env dosyası hazırlanır

Deployment repo içine girilir:

```
None  
cd ~/Desktop/siro-hr-chatbot-deployment
```

.env.example dosyası **.env** olarak kopyalanır:

```
None  
cp .env.example .env
```

Local ortamda örnek **.env** içeriği:

```
None  
VITE_CHATBOT_BACKEND_API=http://localhost:8000  
  
MYSQL_ROOT_PASSWORD=123456  
MYSQL_DATABASE=chatbot_db
```

Adım 4: Docker Compose config kontrolü yapılır

Compose dosyasının geçerli olup olmadığını kontrol etmek için:

```
None  
docker compose config
```

Bu komut, Docker Compose dosyasındaki syntax hatalarını ve environment değişken çözümlmelerini kontrol etmeye yardımcı olur.

Adım 5: İlk temiz başlatma yapılır

İlk testte veya **mysql/init.sql** dosyası değiştirildikten sonra şu komut kullanılmalıdır:

None

```
docker compose down -v  
docker compose up --build
```

Burada `down -v` komutu önemlidir. Çünkü MySQL init script yalnızca MySQL volume ilk kez oluşturulduğunda çalışır. Daha önce `mysql_data` volume'u oluşmuşsa, `init.sql` tekrar otomatik çalışmaz.

`docker compose down -v` şu işlemleri yapar:

- Container'ları durdurur.
- Compose network'ünü kaldırır.
- `mysql_data` volume'unu siler.
- Database'i sıfırlar.
- Bir sonraki `docker compose up --build` sırasında `mysql/init.sql` tekrar çalışır.

Bu komut production ortamında dikkatli kullanılmalıdır. Çünkü `-v` volume'u sildiği için veritabanı verileri kaybolur.

Adım 6: Normal başlatma

Database'i sıfırlamak istemiyorsak normal başlatma için şu komut yeterlidir:

None

```
docker compose up --build
```

Arka planda çalıştırmak için:

None

```
docker compose up --build -d
```

Adım 7: Uygulama kontrol edilir

Backend Swagger arayüzü:

None

```
http://localhost:8000/docs
```

Frontend uygulaması:

None

```
http://localhost:3000
```

MySQL host üzerinden kontrol edilmek istenirse:

None

```
Host: localhost
```

```
Port: 3307
```

```
User: root
```

```
Password: deployment .env içindeki MYSQL_ROOT_PASSWORD
```

```
Database: chatbot_db
```

Adım 8: Container'lar kontrol edilir

Çalışan container'ları görmek için:

None

```
docker ps
```

Beklenen container'lar:

None

```
siro-hr-mysql
```

```
siro-hr-backend
```

```
siro-chatbot-frontend
```

Tüm container'ları görmek için:

None

```
docker ps -a
```

Adım 9: Loglar kontrol edilir

Tüm servis logları:

None

```
docker compose logs -f
```

Backend logları:

None

```
docker compose logs -f backend
```

Frontend logları:

None

```
docker compose logs -f frontend
```

MySQL logları:

None

```
docker compose logs -f mysql
```

Adım 10: Sistemi durdurma

Container'ları durdurmak için:

None

```
docker compose down
```

Database volume'unu da silmek için:

None

```
docker compose down -v
```

Normal kullanımda `docker compose down` tercih edilmelidir. `down -v` yalnızca database'i sıfırlamak veya `init.sql` dosyasını yeniden çalıştırmak gerektiğinde kullanılmalıdır.

14. Kalıcı Veri ve Volume Kullanımı

Sistemde üç önemli kalıcı veri alanı bulunmaktadır:

1. `uploaded_documents`
2. `faiss_index_3b`
3. `mysql_data`

14.1 uploaded_documents

Backend tarafında admin paneli üzerinden yüklenen dokümanlar şu klasörde tutulur:

None

```
siro_hr_chatbot/uploaded_documents/
```

Docker Compose içinde şu şekilde volume olarak bağlanmıştır:

None

```
- ../siro_hr_chatbot/uploaded_documents:/app/uploaded_documents
```

Bu sayede container yeniden build edilse veya silinse bile yüklenen dokümanlar host makinede kalır.

14.2 faiss_index_3b

FAISS index dosyaları şu klasörde tutulur:

None

```
siro_hr_chatbot/faiss_index_3b/
```

Docker Compose içinde şu şekilde bağlanmıştır:

None

```
- ../siro_hr_chatbot/faiss_index_3b:/app/faiss_index_3b
```

Bu klasörde genellikle şu dosyalar bulunur:

None

```
index.faiss  
index.pkl
```

Bu klasör volume olarak bağlandığı için index dosyaları container yaşam döngüsünden bağımsız olarak korunur.

Dokümanlar değiştirildiğinde backend tarafındaki index rebuild mekanizmasına göre FAISS index güncellenmelidir.

14.3 mysql_data

MySQL veritabanı dosyaları Docker named volume içinde tutulur:

None

```
mysql_data:/var/lib/mysql
```

Bu volume sayesinde MySQL container silinse bile veritabanı korunur. Ancak şu komut çalıştırılırsa volume da silinir:

None

```
docker compose down -v
```

Bu durumda database sıfırlanır ve bir sonraki başlatmada `mysql/init.sql` tekrar çalışır.

Bu ayrım önemlidir:

None

```
docker compose down      → Container'ları durdurur, database volume'u korur.
```

```
docker compose down -v   → Container'ları durdurur, database volume'unu siler.
```

Production ortamında `down -v` komutu kullanılmamalıdır veya yalnızca bilinçli veri sıfırlama amacıyla kullanılmalıdır.

15. Production Deployment Yaklaşımı

Local ortamda uygulama `localhost` üzerinden çalışır. Bu beklenen bir durumdur. Ancak production ortamında kullanıcıların `localhost:3000`, `localhost:8000` veya `localhost:3307` gibi adresler görmesi beklenmez.

Production ortamında genellikle şu yapı tercih edilir:

None

```
https://chatbot.siro.com      → frontend container  
https://api-chatbot.siro.com → backend container
```

Bu yapının kurulabilmesi için aşağıdaki adımlar gerekir:

1. Sunucu hazırlanır.
2. Docker ve Docker Compose kurulur.
3. Backend, frontend ve deployment repo dosyaları sunucuya aktarılır veya Git üzerinden çekilir.
4. Production **.env** dosyaları oluşturulur.
5. MySQL password, JWT secret, SAP API bilgileri ve diğer sensitive değerler production ortamına göre ayarlanır.
6. Domain veya subdomain DNS kayıtları sunucuya yönlendirilir.
7. Nginx, Traefik veya benzeri reverse proxy yapılandırılır.
8. SSL sertifikası tanımlanır.
9. Docker Compose ile container'lar ayağa kaldırılır.

Production **.env** Ayarı

Production ortamında deployment repo **.env** dosyasındaki frontend backend API adresi local adres yerine gerçek backend API domain'i olmalıdır:

None

```
VITE_CHATBOT_BACKEND_API=https://api-chatbot.siro.com
```

Bu değişiklikten sonra frontend image yeniden build edilmelidir:

None

```
docker compose up --build -d
```

Çünkü Vite environment değişkenleri runtime sırasında değil, build sırasında frontend koduna gömülür.

Production MySQL Ayarı

Production ortamında MySQL için güçlü ve güvenli bir password kullanılmalıdır:

None

```
MYSQL_ROOT_PASSWORD=<strong-production-password>  
MYSQL_DATABASE=chatbot_db
```

Backend `.env` içinde aynı password kullanılmalıdır:

None

```
DB_HOST=mysql  
DB_PORT=3306  
DB_USER=root  
DB_PASSWORD=<strong-production-password>  
DB_NAME=chatbot_db
```

Production'da root user kullanımı yerine ayrı bir application database user oluşturulması daha güvenli bir yaklaşımdır. Mevcut local/demo yapı root user ile çalışacak şekilde hazırlanmıştır. Kurumsal production ortamında aşağıdaki gibi ayrı bir kullanıcı oluşturulması önerilir:

None

```
MYSQL_USER=chatbot_app  
MYSQL_PASSWORD=<strong-app-password>
```

Bu yaklaşım uygulanacaksa `docker-compose.yml`, `init.sql` ve backend `.env` buna göre güncellenmelidir.

Production Kullanıcı ve Admin Yönetimi

Production ortamında test kullanıcıları veya demo admin hesapları kullanılmamalıdır.

Çalışan kullanıcıların SAP/SSO entegrasyonu üzerinden sağlanması beklenir. Admin kullanıcıların ise şirketin belirleyeceği yetkilendirme politikasına göre atanması gerekir.

Mevcut veritabanı şeması admin kullanıcıları şu tablolarla destekler:

- `users`
- `admins`
- `admin_accounts`

Şu anki uygulama akışında adminlerin manuel SQL insert ile eklenmesi mümkündür. Ancak production için daha kurumsal yaklaşım, SSO role/group bilgisine göre admin yetkisi verilmesidir.

İlk admin kullanıcının nasıl oluşturulacağı şirketin authentication/authorization kararına bağlıdır. Olası yöntemler:

1. İlk adminin manuel SQL insert ile oluşturulması.
2. Admin yetkisinin SSO claim/group bilgisiyle belirlenmesi.
3. İlk admin oluşturulduktan sonra diğer adminlerin admin panel üzerinden yönetilmesi.

Bu karar production deployment öncesinde netleştirilmelidir.

Reverse Proxy Mantığı

Production ortamında container portları doğrudan kullanıcılara açılmak zorunda değildir. Bunun yerine reverse proxy kullanılır.

Örnek frontend akışı:

```
None
Kullanıcı
  ↓
https://chatbot.siro.com
  ↓
Nginx Reverse Proxy
  ↓
frontend container:80
```

Örnek backend akışı:

```
None
Kullanıcı / Frontend
  ↓
https://api-chatbot.siro.com
  ↓
Nginx Reverse Proxy
  ↓
backend container:8000
```

Bu sayede kullanıcı tarayıcıda `localhost:3000` veya `localhost:8000` görmez. Bunun yerine kurum tarafından belirlenen domain veya subdomain adreslerini görür.

16. CPU ve GPU Değerlendirmesi

Local Docker testinde backend CPU üzerinde çalışabilir. Ancak cevap süreleri uzun olabilir. Bunun sebebi backend'in birden fazla model bileşeni yüklemesi ve çalıştırmasıdır:

- BGE-M3 embedding modeli
- BGE reranker modeli
- Qwen2.5-3B-Instruct LLM
- mDeBERTa tabanlı kategori sınıflandırma modeli
- FAISS index
- RAG pipeline

Demo ve düşük trafik senaryolarında CPU ile çalışmak mümkün olabilir. Ancak production veya production'a yakın performans için GPU destekli bir sunucu önerilir.

Önerilen minimum GPU destekli yapı:

None

GPU: NVIDIA T4 / L4 / A10 / RTX serisi veya eşdeğer

VRAM: Minimum 8 GB, tercihen 12-16 GB

CPU: 4-8 vCPU

RAM: 16-32 GB

Disk: 200-500 GB SSD

OS: Ubuntu Server 22.04 / 24.04

GPU kullanılacaksa sunucuda ayrıca şu bileşenlerin bulunması gerekir:

- NVIDIA driver
- NVIDIA Container Toolkit
- CUDA uyumlu PyTorch kurulumu
- Docker GPU runtime ayarları

GPU destekli deployment için `docker-compose.yml` dosyasına ek GPU ayarları eklenmesi gerekebilir.

17. Kullanışlı Docker Komutları

Sistemi build edip başlatmak:

None

```
docker compose up --build
```

Sistemi arka planda başlatmak:

None

```
docker compose up --build -d
```

Sistemi durdurmak:

None

```
docker compose down
```

Sistemi durdurmak ve MySQL volume dahil tüm volume'ları silmek:

None

```
docker compose down -v
```

`down -v` komutu database'i sıfırlar. Bu komut yalnızca bilinçli olarak kullanılmalıdır.

Çalışan container'ları görmek:

None

```
docker ps
```

Tüm container'ları görmek:

None

```
docker ps -a
```

Tüm servis loglarını görmek:

None

```
docker compose logs -f
```

Sadece backend loglarını görmek:

None

```
docker compose logs -f backend
```

Sadece frontend loglarını görmek:

None

```
docker compose logs -f frontend
```

Sadece MySQL loglarını görmek:

None

```
docker compose logs -f mysql
```

Cache kullanmadan yeniden build etmek:

None

```
docker compose build --no-cache
```

Compose dosyasını doğrulamak:

None

```
docker compose config
```

Gereksiz Docker dosyalarını temizlemek:

None

```
docker system prune
```

Belirli isimde kalmış eski container'ı silmek:

None

```
docker rm -f siro-hr-backend  
docker rm -f siro-chatbot-frontend  
docker rm -f siro-hr-mysql
```

18. Karşılaşılan Hatalar ve Çözümleri

Hata: `docker: command not found`

Sebebi:

Docker kurulu değildir veya terminal tarafından tanınmamaktadır.

Çözüm:

- Docker Desktop kurulmalıdır.
- Docker Desktop başlatılmalıdır.
- Terminal yeniden açılmalıdır.
- Aşağıdaki komutla kontrol yapılmalıdır:

```
docker --version
```

Hata: Eksik Python paketi

Örnek hata:

None

```
ModuleNotFoundError: No module named 'docx'
```

Sebebi:

Kodda kullanılan bir Python paketi `requirements.txt` içinde yer almıyor olabilir.

Çözüm:

Eksik paket `requirements.txt` dosyasına eklenmelidir. `docx` için doğru pip paketi `python-docx` paketidir.

Daha sonra yeniden build alınır:

None

```
docker compose up --build
```

Hata: Frontend TypeScript build hatası

Örnek hata:

None

```
TS6133: declared but its value is never read
```

Sebebi:

TypeScript kullanılmayan değişken veya importları hata olarak değerlendiriyor olabilir.

Çözüm:

- Kullanılmayan değişkenler veya importlar temizlenmelidir.
- Önce local olarak build denenmelidir:

```
npm run build
```

- Sonra Docker build tekrar çalıştırılmalıdır:

```
docker compose up --build
```

Hata: Port zaten kullanımda

Örnek hata:

None

```
port is already allocated
```

Sebebi:

Aynı portu kullanan başka bir container veya process çalışıyor olabilir.

Çözüm:

None

```
docker ps
```

```
docker stop <container_id>
```

Alternatif olarak `docker-compose.yml` içindeki host port değiştirilebilir.

Hata: Container name conflict

Örnek hata:

None

```
Conflict. The container name "/siro-hr-backend" is already in use.
```

Sebebi:

Aynı isimde eski bir container hâlâ sistemde duruyordur. Bu durum daha önce `docker run` ile veya eski Compose dosyasıyla container oluşturulduysa görülebilir.

Çözüm:

Önce SIRO ile ilgili container'lar listelenir:

None

```
docker ps -a | grep siro
```

Sonra eski container'lar temizlenir:

None

```
docker rm -f siro-hr-backend siro-chatbot-frontend siro-hr-mysql
```

Daha sonra tekrar çalıştırılır:

None

```
docker compose up --build
```

Hata: Backend MySQL'e bağlanamıyor

Olası sebepler:

- Backend `.env` içinde `DB_HOST=localhost` kalmış olabilir.
- Backend `DB_PASSWORD`, MySQL `MYSQL_ROOT_PASSWORD` ile aynı değildir.
- MySQL container henüz hazır değildir.
- `mysql_data` volume eski ve hatalı bir state içeriyor olabilir.
- `init.sql` ilk çalıştırmada hata almış olabilir.

Çözüm:

Backend `.env` kontrol edilir:

```
None
DB_HOST=mysql
DB_PORT=3306
DB_USER=root
DB_PASSWORD=123456
DB_NAME=chatbot_db
```

Deployment `.env` kontrol edilir:

```
None
MYSQL_ROOT_PASSWORD=123456
MYSQL_DATABASE=chatbot_db
```

MySQL logları incelenir:

```
None
docker compose logs -f mysql
```

İlk test için database sıfırlanıp init script yeniden çalıştırılabilir:

```
None
docker compose down -v
docker compose up --build
```

Hata: `init.sql` değişti ama database güncellenmedi

Sebebi:

MySQL Docker image'ı `/docker-entrypoint-initdb.d/` altındaki scriptleri yalnızca ilk database initialization sırasında çalıştırır. Eğer `mysql_data` volume daha önce oluştuysa, `init.sql` değişse bile otomatik tekrar çalışmaz.

Çözüm:

Local test ortamında volume sıfırlanır:

None

```
docker compose down -v  
docker compose up --build
```

Production ortamında `down -v` kullanılmamalıdır. Production'da schema değişiklikleri migration mantığıyla uygulanmalıdır.

Hata: Frontend backend'e erişemiyor

Olası sebepler:

- Backend container çalışmıyor olabilir.
- Frontend yanlış backend API adresiyle build edilmiş olabilir.
- CORS ayarları eksik olabilir.
- Backend domain veya port bilgisi yanlış olabilir.

Çözüm:

Deployment `.env` dosyası kontrol edilir:

None

```
VITE_CHATBOT_BACKEND_API=http://localhost:8000
```

Production için:

None

```
VITE_CHATBOT_BACKEND_API=https://api-chatbot.siro.com
```

Frontend image yeniden build edilmelidir:

None

```
docker compose up --build
```

Backend Swagger kontrol edilmelidir:

None

```
http://localhost:8000/docs
```

Backend logları incelenmelidir:

None

```
docker compose logs -f backend
```

Hata: Model yükleme çok uzun sürüyor

Sebebi:

Backend embedding, reranker, LLM ve classifier modellerini yüklüyor olabilir. İlk çalıştırmada Hugging Face üzerinden model indirme işlemi yapılabilir. CPU ortamlarında model yükleme ve cevap üretimi daha yavaştır.

Çözüm:

- Model yüklemenin tamamlanması beklenmelidir.
- Production için GPU destekli ortam tercih edilmelidir.
- Hugging Face rate limit durumları için token tanımlanabilir.
- Backend loglarında application startup tamamlanana kadar beklenmelidir.

19. Bakım ve Güncelleme Süreci

Backend kodu güncellendiğinde

Backend kodunda değişiklik yapıldıktan sonra sistem yeniden build edilmelidir:

None

```
docker compose up --build -d
```

Eğer `requirements.txt` değiştiyse yeniden build zorunludur.

Frontend kodu güncellendiğinde

Frontend kodu değiştirildiğinde image yeniden build edilmelidir:

None

```
docker compose up --build -d
```

Backend API URL değiştirildiğinde

Deployment repo `.env` dosyasındaki değer değiştirilmelidir:

None

```
VITE_CHATBOT_BACKEND_API=https://api-chatbot.siro.com
```

Daha sonra frontend yeniden build edilmelidir:

None

```
docker compose up --build -d
```

Çünkü Vite environment değişkenleri build sırasında frontend koduna gömülür.

Backend `.env` değiştiğinde

Backend runtime değişkenleri değiştirildiğinde container yeniden başlatılmalıdır:

None

```
docker compose down  
docker compose up --build -d
```

Eğer sadece `.env` değişti ve image rebuild gerekmiyorsa şu da kullanılabilir:

None

```
docker compose up -d
```

Ancak build gerektiren kod/dependency değişiklikleri varsa `--build` kullanılmalıdır.

MySQL schema değiştiğinde

Local test ortamında `mysql/init.sql` değiştirildiyse ve yeni schema'nın baştan uygulanması isteniyorsa:

None

```
docker compose down -v  
docker compose up --build
```

Bu işlem database'i sıfırlar.

Production ortamında schema değişiklikleri için `down -v` kullanılmamalıdır. Bunun yerine migration scriptleri veya kontrollü SQL update dosyaları kullanılmalıdır.

Dokümanlar güncellendiğinde

Yüklenen dokümanlar şu klasörde tutulur:

None

```
siro_hr_chatbot/uploaded_documents/
```

Bu klasör volume olarak bağlandığı için container silinse bile dosyalar korunur.

FAISS index güncellendiğinde

FAISS index dosyaları şu klasörde tutulur:

None

```
siro_hr_chatbot/faiss_index_3b/
```

Dokümanlar değiştiğinde backend uygulamasının index güncelleme veya rebuild mekanizması çalıştırılmalıdır.

20. Final Kontrol Listesi

Deployment başarılı kabul edilmeden önce aşağıdaki kontroller yapılmalıdır:

- Docker kurulu ve çalışıyor.
- Docker Compose komutu çalışıyor.
- Backend repo güncel.
- Frontend repo güncel.
- Deployment repo güncel.
- Üç repo aynı üst klasörde bulunuyor.

- Deployment repo içinde `.env.example` var.
- Deployment repo içinde local `.env` oluşturulmuş.
- Deployment `.env` Git'e pushlanmıyor.
- Backend `.env` dosyası backend klasöründe mevcut.
- Backend `.env` Git'e pushlanmıyor.
- `MYSQL_ROOT_PASSWORD` ve backend `DB_PASSWORD` aynı.
- Backend `DB_HOST=mysql` olarak ayarlanmış.
- `mysql/init.sql` mevcut.
- `docker-compose.yml` içinde MySQL servisi mevcut.
- `mysql_data` volume tanımlı.
- `uploaded_documents` volume olarak bağlanmış.
- `faiss_index_3b` volume olarak bağlanmış.
- İlk test için `docker compose down -v` çalıştırılmış.
- `docker compose up --build` komutu üç servisi de ayağa kaldırıyor.
- MySQL container çalışıyor.
- Backend container çalışıyor.
- Frontend container çalışıyor.
- Backend Swagger `http://localhost:8000/docs` adresinden açılıyor.
- Frontend `http://localhost:3000` adresinden açılıyor.
- Frontend backend'e istek gönderebiliyor.
- Login işlemi çalışıyor.
- Chat oluşturma çalışıyor.
- Mesaj gönderme çalışıyor.
- Admin panel açılıyor.
- Doküman listeleme/yükleme/silme çalışıyor.
- Kategori kısıtlama ekranı çalışıyor.
- `chatbot_categories` tablosu init sırasında doluyor.
- MySQL verileri `mysql_data` volume içinde kalıcı tutuluyor.
- Production için test kullanıcı/admin seed verilerinin kullanılmayacağı netleştirilmiş.
- Production için SAP/SSO kullanıcı yönetimi planı netleştirilmiş.
- Production için admin yetkilendirme yöntemi netleştirilmiş.
- Production için domain/reverse proxy/SSL planı netleştirilmiş.
- Production için güvenli MySQL password ve secret değerleri belirlenecek.
- GPU ihtiyacı ve minimum sunucu gereksinimleri değerlendirilmiş.

Bu kontroller tamamlandığında SIRO HR Chatbot uygulaması backend, frontend ve MySQL dahil Dockerize edilmiş; local test ve production deployment için hazır hale getirilmiş kabul edilebilir.

21. Teslim Notu

Bu deployment yapısı local test ortamında backend, frontend ve MySQL'i tek komutla ayağa kaldıracak şekilde hazırlanmıştır:

None

```
docker compose up --build
```

MySQL schema initialization işlemi `mysql/init.sql` dosyası ile otomatikleştirilmiştir. Ancak `init.sql` yalnızca MySQL volume ilk kez oluşturulduğunda çalışır. Bu nedenle local testte schema değişiklikleri sonrası şu komut kullanılmalıdır:

None

```
docker compose down -v  
docker compose up --build
```

Production ortamında `down -v` komutu veri kaybına yol açacağı için kullanılmamalıdır. Production schema değişiklikleri kontrollü migration süreciyle yönetilmelidir.

Kullanıcı ve admin hesapları production ortamında test seed verileriyle oluşturulmamalıdır. Çalışan kullanıcıların SAP/SSO entegrasyonu üzerinden sağlanması, admin yetkilendirmesinin ise şirketin belirleyeceği politikaya göre yapılması önerilir.